

Applied Deep Learning

First Assignment

Diogo Vasco	2023226969
Ulyana Tsitova	2026179126

10/2026

1 Part I – CNN on CIFAR-10

1.1 Data and architecture

CIFAR-10 (50 000 training and 10 000 test RGB images of $3 \times 32 \times 32$, 10 classes) was loaded from Hugging Face (`uoft-cs/cifar10`, identical to the official release). The training set was split 80/20 into training (40 000) and validation (10 000) images with a fixed seed (42); the test set was used only for the final evaluation. Images were normalized with the CIFAR-10 channel statistics, and only the training images were augmented (random crop with 4-pixel padding, horizontal flip).

The baseline provided in the assignment (64% mCA, $lr = 0.0001$, 25 epochs) keeps the full 32×32 resolution and flattens 16 384 values, so almost all of its 4.22 M parameters are in the first fully-connected layer, while its feature extractor stays shallow. The proposed network (Fig. 1) uses the maximum of six convolutional layers in three VGG-style blocks, each with two 3×3 convolutions (Batch Normalization and ReLU) followed by 2×2 max-pooling and dropout (0.1, 0.2, 0.3). The channels double (64, 128, 256) while the resolution halves ($32 \to 16 \to 8 \to 4$); the classifier has one hidden layer (256 neurons, ReLU, dropout 0.4) and 10 outputs, giving 2.20 M parameters, about half of the baseline. Stacked 3×3 convolutions enlarge the receptive field with fewer parameters, Batch Normalization stabilizes training with large learning rates, max-pooling strongly reduces the fully-connected part, and dropout, weight decay and augmentation reduce overfitting.

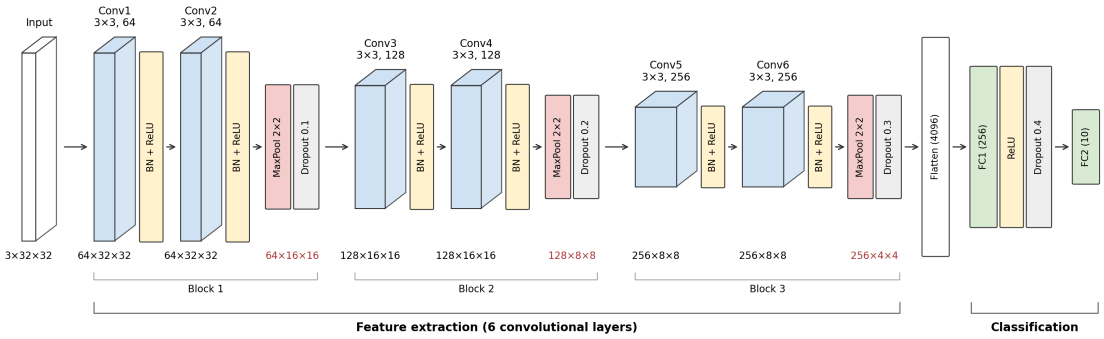


Figure 1: Proposed CNN architecture for CIFAR-10 (6 convolutional layers, 2 fully-connected layers).

1.2 Training and results

All configurations were trained for 30 epochs with the cross-entropy loss, weight decay $5 \cdot 10^{-4}$ and a cosine annealing scheduler; SGD used momentum 0.9. The models were evaluated on the test set with mCA, macro Precision, Recall and F1-Score (scikit-learn) and the confusion matrix; since the test set is balanced, macro Recall equals mCA. The best configuration was selected by validation mCA. Four combinations of optimizer, learning rate and batch size were compared (Table 1), and all of them exceeded the required 85% mCA.

Config 1 (SGD, $lr = 0.05$) started unstably (0.28–0.32 validation mCA in the first two epochs, oscillating validation loss), but as the cosine schedule lowered the learning rate it reached the best test mCA (90.15%). **Config 2** ($lr = 0.005$) learned fastest at first but ended lowest (86.94%)

Table 1: Hyperparameter configurations and results on CIFAR-10 (validation mCA: best epoch; other metrics: test set).

Config	Optimizer	LR	Batch	Val mCA	Test mCA	Precision	Recall	F1
Baseline	–	0.0001	–	–	0.640	–	–	–
C1	SGD	0.05	128	0.9053	**0.9015**	0.9011	**0.9015**	**0.9011**
C2	SGD	0.005	128	0.8801	0.8694	0.8689	0.8694	0.8687
C3	SGD	0.05	512	0.8818	0.8755	0.8753	0.8755	0.8750
C4	Adam	0.001	128	**0.9064**	0.9012	0.9008	0.9012	0.9008

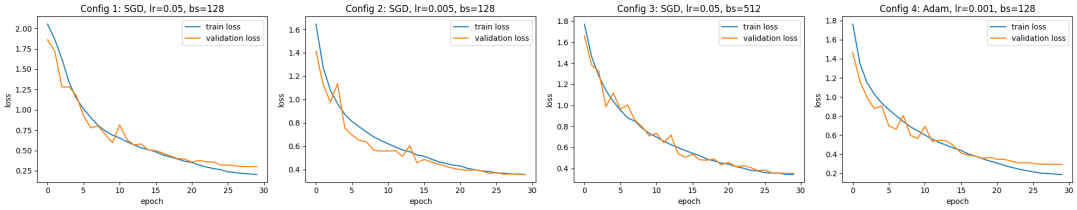


Figure 2: Training and validation loss curves of the four hyperparameter configurations.

with equal training and validation losses (0.357): mild *underfitting*, as a ten times smaller learning rate was not enough in 30 epochs. **Config 3** (batch 512) makes four times fewer updates per epoch (79 vs. 313) and also underfits (losses 0.342 vs. 0.353, 87.55%). **Config 4** (Adam) gave the smoothest curves and practically the same result as Config 1 (90.12%). Both show a gap between training and validation loss (0.188 vs. 0.293 and 0.204 vs. 0.300), i.e. mild *overfitting*, but the validation loss only flattens, so the regularization prevents stronger overfitting. In the first epochs the validation loss is below the training loss in all runs, because dropout and augmentation are active only during training. Config 4 had the highest validation mCA and was selected; on the test set it equals Config 1 (0.03 pp difference) and improves the baseline by about 26 pp with half the parameters.

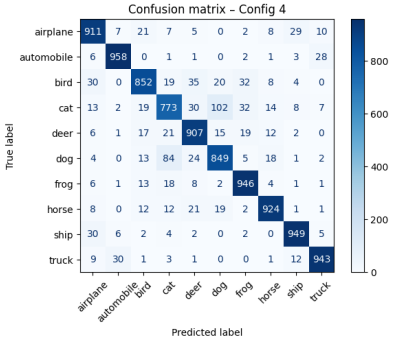


Figure 3: Confusion matrix of the best configuration (Config 4) on the CIFAR-10 test set.

The confusion matrix (Fig. 3) shows the best results for automobile (95.8%), ship (94.9%), frog (94.6%) and truck (94.3%). The hardest class is cat (77.3%), mostly confused with dog (102 cats as dogs, 84 dogs as cats), since at 32×32 pixels their similar shapes and fur are hard to separate. Airplane and ship (29/30 errors, uniform blue background) and automobile and truck (28/30 errors, similar shape) are also often confused.

2 Part II – ResNet-18 and MobileNetV3 on STL-10

2.1 Dataset and training protocol

STL-10 contains 13 000 labelled $3 \times 96 \times 96$ RGB images across 10 classes, officially split into 5 000 training and 8 000 test images. The training set was split 80/20 into training (4 000) and validation (1 000) with `random_split` (seed 42); the test set was used only for the final evaluation. Images were resized to 224×224 (input size of the ImageNet backbones) and normalized with the standard ImageNet statistics. No data augmentation was applied, so the four single-backbone configurations differ only in the backbone and the initialization.

The two backbones are **ResNet-18** (11.7 M params, residual blocks) and **MobileNetV3-Large** (5.5 M params, depthwise separable convolutions, squeeze-and-excitation blocks, Hardswish activation). They were chosen as representatives of two different design philosophies that both reach state-of-the-art ImageNet accuracy with a moderate parameter count. For each backbone, the final classification layer was replaced by a linear layer with 10 output neurons, and each backbone was trained from scratch (`weights=None`) and from ImageNet-pretrained weights (fine-tuning the whole network), giving four directly comparable models.

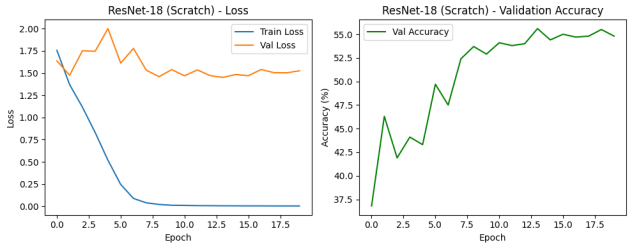
All four models were trained with the same hyperparameters: Adam ($lr = 10^{-4}$, weight decay 10^{-4}), cross-entropy loss, batch size 64, 20 epochs. The evaluation protocol matches Part I (mCA, macro Precision/Recall/F1, confusion matrix); macro Recall equals mCA because the test set is balanced (800 images per class).

2.2 Results

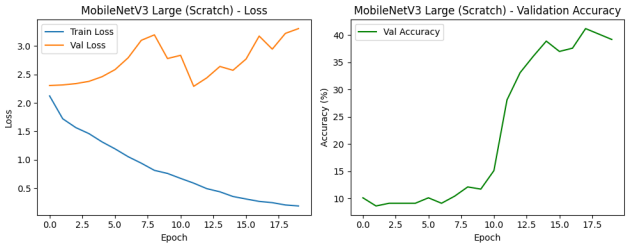
Table 2 summarizes the test-set results of the four single-backbone models and of the middle fusion model (Section 2.3). The two pretrained models reach around 95–96% mCA, while the two models trained from scratch stay below 60%, confirming the well-known difficulty of training large networks on only 4 000 images. Figure 4 shows the corresponding loss curves and Fig. 5 the confusion matrices.

Table 2: Classification results on the STL-10 test set (macro-averaged metrics, in %).

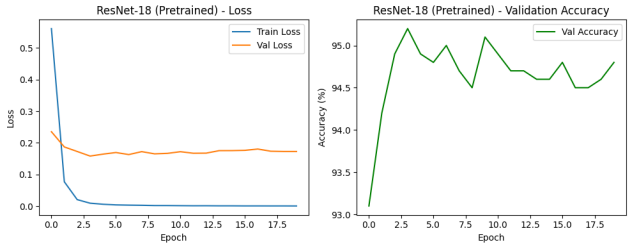
Model	mCA	Precision	Recall	F1
ResNet-18 (Scratch)	56.89	56.93	56.89	56.70
MobileNetV3-L (Scratch)	39.17	40.60	39.17	38.83
ResNet-18 (Pretrained)	95.41	95.41	95.41	95.40
MobileNetV3-L (Pretrained)	95.72	95.73	95.72	95.72
Middle Fusion	**96.04**	**96.03**	**96.04**	**96.03**



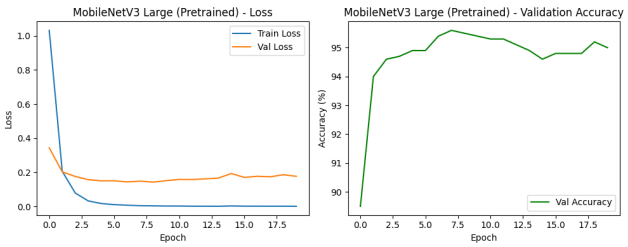
(a) ResNet-18 (Scratch)



(b) MobileNetV3-L (Scratch)

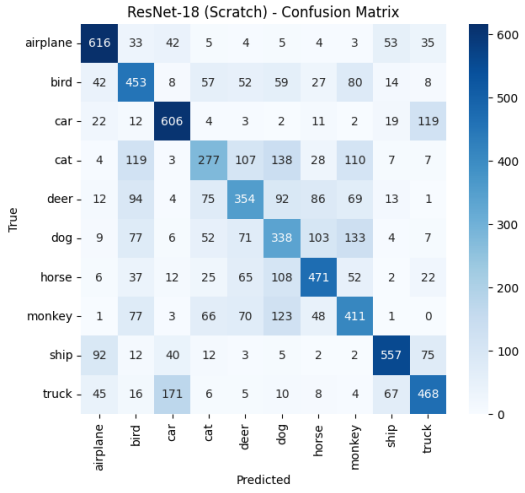


(c) ResNet-18 (Pretrained)

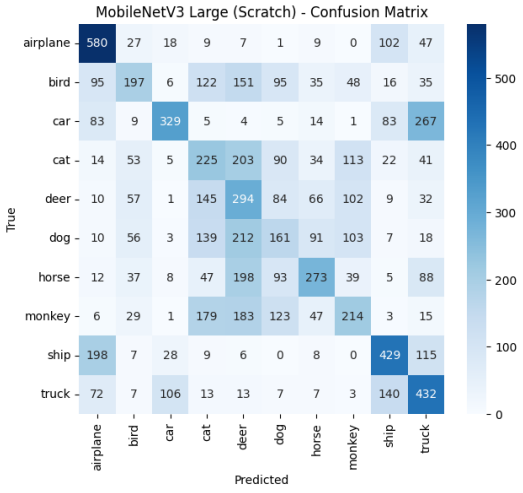


(d) MobileNetV3-L (Pretrained)

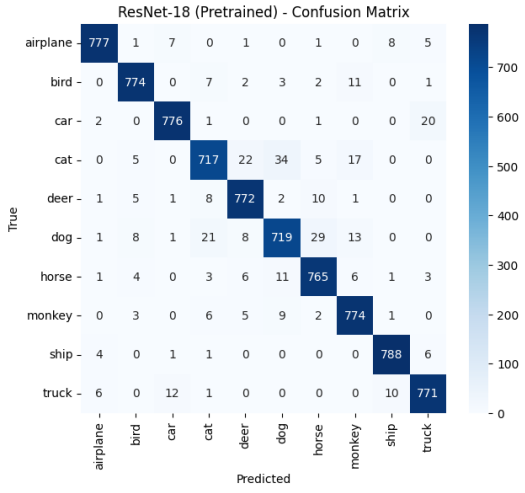
Figure 4: Training and validation loss curves of the four Part II single-backbone models.



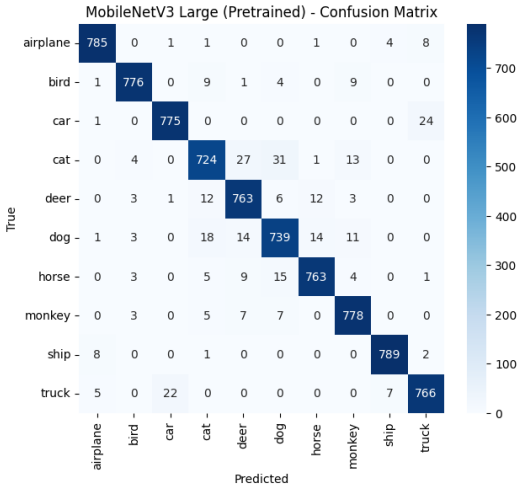
(a) ResNet-18 (Scratch)



(b) MobileNetV3-L (Scratch)



(c) ResNet-18 (Pretrained)



(d) MobileNetV3-L (Pretrained)

Figure 5: Confusion matrices of the four Part II single-backbone models on the STL-10 test set.

2.3 Middle fusion

Middle fusion combines the two pretrained backbones at the level of their intermediate feature maps (Fig. 6). Both are used as frozen feature extractors (`requires_grad = False`); only the final linear layer is trained, adding $1\,472 \times 10 + 10 = 14\,730$ parameters. Training uses the same hyperparameters as the single backbones. The fusion reaches 96.04% mCA, 0.3 percentage points above the best single backbone.

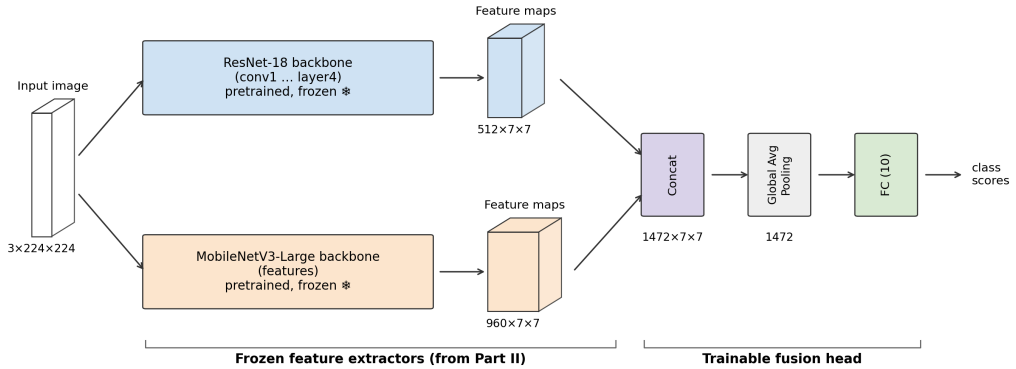
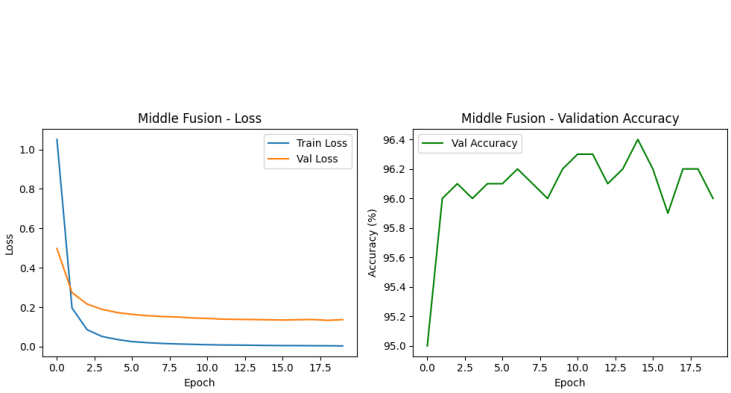
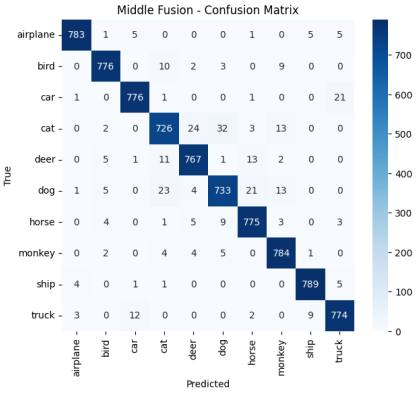


Figure 6: Middle fusion architecture: both backbones are frozen; their $512 \times 7 \times 7$ and $960 \times 7 \times 7$ feature maps are concatenated channel-wise ($1\,472 \times 7 \times 7$), globally average pooled and classified by a single $1\,472 \to 10$ linear layer.



(a) Loss and validation accuracy



(b) Confusion matrix

Figure 7: Middle fusion model: training curves and confusion matrix on the STL-10 test set.

2.4 Analysis

Scratch vs. pretrained. The scratch models stay far below their pretrained counterparts (−38.5 pp for ResNet-18, −56.6 pp for MobileNetV3-L): 4 000 training images are not enough to learn generic low-level features (edges, textures, colour patterns), which ImageNet-pretrained weights supply for free, so fine-tuning only needs to adapt the higher layers. From scratch, ResNet-18 (56.89%) is far above MobileNetV3-L (39.17%): the plain residual blocks optimize easily on a small dataset, whereas MobileNetV3 depends on components (SE attention, Hardswish, inverted bottlenecks) that need a large dataset to be calibrated and essentially fail to train here: its scratch confusion matrix (Fig. 5b) is visibly spread out, with several classes rarely predicted,

while the two pretrained matrices are almost diagonal. In the pretrained setting the ordering reverses: MobileNetV3-L (95.72%) is slightly ahead of ResNet-18 (95.41%) despite having half the parameters, thanks to its stronger architectural priors and more recent ImageNet recipe.

Fusion and confusions. The middle fusion improves the best single backbone by 0.3 percentage points at a very small cost (one linear layer): the two backbones fail on different images, so their concatenated features give the classifier complementary cues. The confusion matrix (Fig. 7b) shows that the residual errors concentrate in the *animal* classes (cat/dog, deer/horse), which share shapes, poses and natural backgrounds, while the vehicle classes are recognised much more reliably.

3 Part III – Image Retrieval on STL-10

3.1 Retrieval pipeline

The best Part II models (ImageNet-pretrained ResNet-18 and MobileNetV3-Large fine-tuned on STL-10) were turned into embedding networks by replacing their final classification layer with an identity mapping: the ResNet-18 embedding is the 512-dimensional output of the global average pooling, and the MobileNetV3-L embedding the 1 280-dimensional output of the penultimate fully-connected layer. The test images were preprocessed as in Part II, and the embeddings were extracted in evaluation mode.

The STL-10 test set (800 images per class) was split independently within each class with a fixed seed (42) into 160 query and 640 gallery images, giving 1 600 query and 6 400 non-overlapping gallery images. Each query embedding $\mathbf{q}$ is compared with all gallery embeddings $\mathbf{g}$ by the cosine similarity

$$\mathrm{sim}(\mathbf{q}, \mathbf{g}) = \frac{\mathbf{q} \cdot \mathbf{g}}{\|\mathbf{q}\| \, \|\mathbf{g}\|}; \tag{1}$$

after L2-normalization, the full $1\,600 \times 6\,400$ similarity matrix is a single matrix product. For every query, the gallery is sorted by descending similarity, which gives the full ranking (used for the Average Precision) and the Top-5 result. The class labels are not used during retrieval; they were only stored for the evaluation and the metric-learning training.

3.2 Retrieval with the classification embeddings

The retrieval quality was measured with three complementary metrics computed from the full ranking of the gallery. A gallery image is considered *relevant* to a query if it belongs to the same class.

Precision at K measures the fraction of relevant images among the first K retrieved results, averaged over all queries:

$$\mathrm{P@}K = \frac{1}{N_q} \sum_{i=1}^{N_q} \frac{1}{K} \sum_{k=1}^{K} 1\big[y_{i,k}^{\mathrm{ret}} = y_i^{\mathrm{query}}\big] \,, \tag{2}$$

where N_q is the number of queries and $y_{i,k}^{\mathrm{ret}}$ is the class label of the k-th image retrieved for query i. P@1 measures the single best match, P@5 the quality of the Top-5 shortlist.

Mean Average Precision (mAP) summarizes the full ranking. For each query, the Average Precision is the average of the precisions computed at the ranks of all relevant gallery images:

$$\mathrm{AP}_i = \frac{1}{R_i} \sum_{k=1}^{N_g} 1\big[y_{i,k}^{\mathrm{ret}} = y_i^{\mathrm{query}}\big]\, \mathrm{P@}k, \tag{3}$$

where $R_i = 640$ is the number of relevant gallery images per query and N_g is the gallery size; the mAP is the mean of the AP_i. Unlike P@K, it depends on the position of *every* relevant image, and therefore reflects how well the embedding space groups a whole class together.

All three metrics were computed in a vectorised way from the full $1\,600 \times 6\,400$ similarity matrix: a binary relevance mask was built from the ranked gallery labels, then reduced to P@K (first K columns) and AP_i (cumulative sum normalised by rank). Results for the classification embeddings are in the first two rows of Table 3.

Table 3: Retrieval results on the STL-10 test set (1 600 queries, 6 400 gallery images).

Embeddings	P@1	P@5	mAP
ResNet-18 (classification)	0.949	0.944	0.921
MobileNetV3-L (classification)	0.961	0.953	0.908
ResNet-18 (Triplet, $m = 1.0$)	0.941	0.941	**0.938**
MobileNetV3-L (Triplet, $m = 1.0$)	0.938	0.938	**0.940**

3.3 Metric learning with Triplet Loss

The classification embeddings are optimised for discrimination (correct class on top of the softmax), not for the geometry of the embedding space. Metric learning optimises that geometry directly, and of the two options suggested in the assignment *Triplet Loss* was implemented. For every anchor $\mathbf{a}$, a positive $\mathbf{p}$ is sampled from the same class and a negative $\mathbf{n}$ from any other class; embeddings are L2-normalised and compared with the Euclidean distance, enforcing

$$\|f(\mathbf{a}) - f(\mathbf{p})\|_2 + m \;<\; \|f(\mathbf{a}) - f(\mathbf{n})\|_2, \tag{4}$$

with margin m. The loss is $\max(0,\, d(\mathbf{a},\mathbf{p}) - d(\mathbf{a},\mathbf{n}) + m)$ and is zero as soon as the triplet already satisfies the margin.

Triplet sampling. Triplets are built on-the-fly from the whole STL-10 training split (all 5 000 images; unlike Part II, no validation subset is held out, since the backbones are only fine-tuned and not model-selected here): the anchor comes from the `DataLoader`, the positive is drawn uniformly from the other images of its class, and the negative from a uniformly drawn different class (random triplets, no auxiliary network).

Initialization. As allowed by the assignment, the two Part II pretrained models are used as initialization. The classification layer is replaced by `nn.Identity`, so the backbone outputs the 512-d (ResNet-18) or 1 280-d (MobileNetV3-L) embedding, and the whole backbone is fine-tuned under the triplet objective.

Hyperparameters. Embeddings are L2-normalised, so the triplet loss operates on the same geometry as the retrieval evaluation (cosine ranking is equivalent to Euclidean ranking after normalisation). The implementation uses `nn.TripletMarginLoss` ($p = 2$) with Adam ($lr = 10^{-4}$,

weight decay 10^{-4}), batch size 64 and 10 epochs. The margin $m = 1.0$ was chosen after a short sweep over $\{0.2, 1.0, 2.0\}$: $m = 0.2$ collapses to near-zero loss at epoch 1 (the Part II features already satisfy such a small margin, so almost no gradient flows), $m = 2.0$ plateaus well above zero because `TripletMarginLoss` uses the non-squared Euclidean distance, which for L2-normalised embeddings is bounded by 2, so the constraint $d(\mathbf{a}, \mathbf{p}) + 2 < d(\mathbf{a}, \mathbf{n})$ can only be satisfied in the degenerate case $d(\mathbf{a}, \mathbf{p}) = 0$, $d(\mathbf{a}, \mathbf{n}) = 2$. The intermediate value $m = 1.0$ gives smooth loss curves (Fig. 8) and the best overall mAP. It should be noted that this sweep was evaluated with the retrieval metrics computed on the test query/gallery split, since no separate validation split was defined for retrieval; the reported gains for $m = 1.0$ are therefore mildly optimistic, and a cleaner protocol would select the margin on a held-out validation split carved out of the training set.

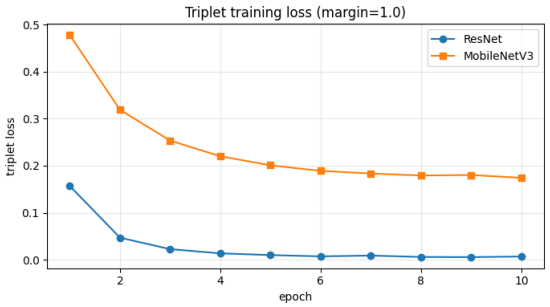


Figure 8: Triplet training loss over the 10 fine-tuning epochs for ResNet-18 and MobileNetV3-L ($m = 1.0$). Both curves drop smoothly and plateau above zero, showing that the optimizer is still finding violating triplets.

After fine-tuning, query and gallery embeddings were extracted with the triplet-trained backbones on exactly the same split as in Section 3.1, and the three retrieval metrics were recomputed (last two rows of Table 3).

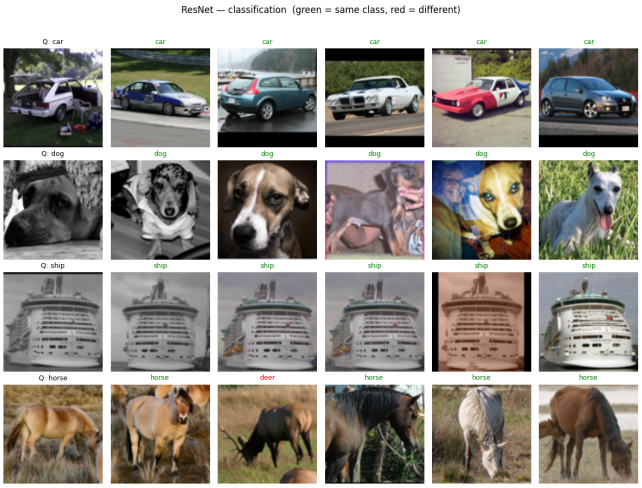
3.4 Qualitative results

Figure 9 shows four random queries and their Top-5 results for each backbone and objective (green border: same class, red: different). Two successful cases are the ship query, for which all four embeddings return five ships, mostly the same cruise ship seen from a similar side viewpoint and in similar grey tones, and the car query with ResNet-18 (Fig. 9a,b), which returns only cars with a similar side view. Two unsuccessful cases come from MobileNetV3-L: for the car query, three of the five classification results are trucks (Fig. 9c), which share the boxy shape and viewpoint of the query; for the dog query, the classification embedding retrieves a cat and a horse, and the triplet embedding a horse and a monkey (Fig. 9c,d), all close-ups of dark-furred animal heads. Shape, fur texture, colour and framing can therefore outweigh the class identity. With the triplet-trained embeddings the number of out-of-class images in these Top-5 lists drops for both backbones (from 1 to 0 for ResNet-18 and from 5 to 3 for MobileNetV3-L), consistent with the mAP gain; occasionally an atypical same-class image ranked first by the classification embedding is displaced by a more prototypical one, which explains the small loss in P@1.

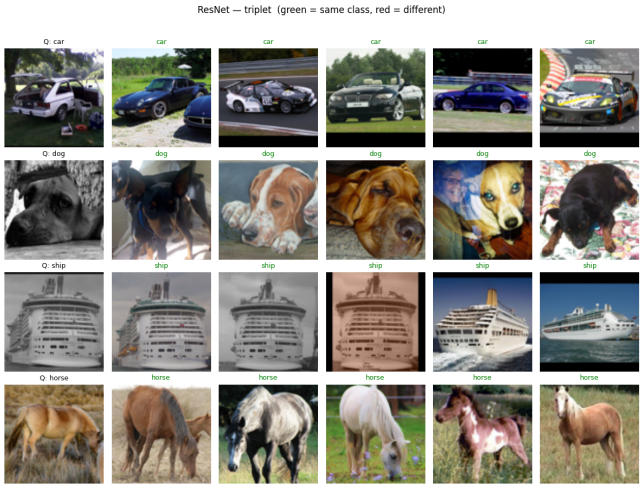
3.5 Discussion

Does the best classifier give the best retrieval? Not strictly. MobileNetV3-L was the better classifier (95.72% vs. 95.41% mCA) and also leads on P@1 and P@5 (0.961/0.953 vs. 0.949/0.944), but on mAP the order reverses (0.908 vs. 0.921). Cross-entropy only requires the correct class on top of the softmax, not tightly packed intra-class samples: the best classifier has the best *top of the list*, not necessarily the best *global organisation* of the embedding space.

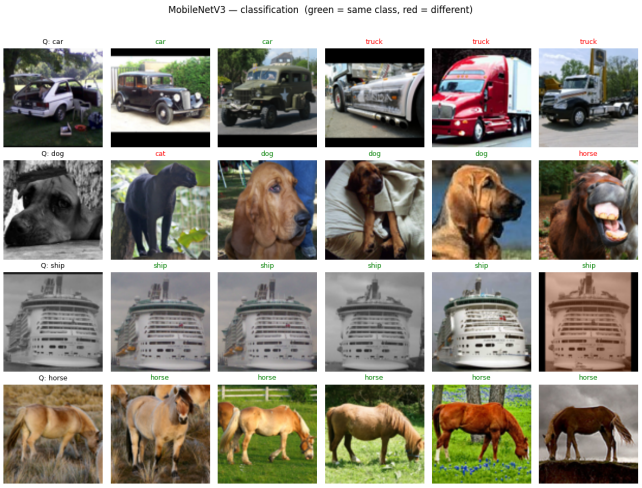
Does Triplet Loss improve retrieval? Yes for mAP, slightly worse for P@1, on both backbones (Table 3): ResNet-18 gains +1.7 pp mAP and loses −0.8 pp P@1; MobileNetV3-L gains +3.3 pp mAP and loses −2.3 pp P@1. Triplet loss optimises exactly the geometry evaluated at retrieval time, packing each class into a tighter cluster so the *entire* ranked gallery contains fewer out-of-class interlopers, at the cost of a small drop in top-1 precision where the classification head already excelled. The gain is larger for MobileNetV3-L, which had the stronger classifier but the weaker retrieval representation; after fine-tuning the two are essentially equivalent on mAP (0.938 vs. 0.940).



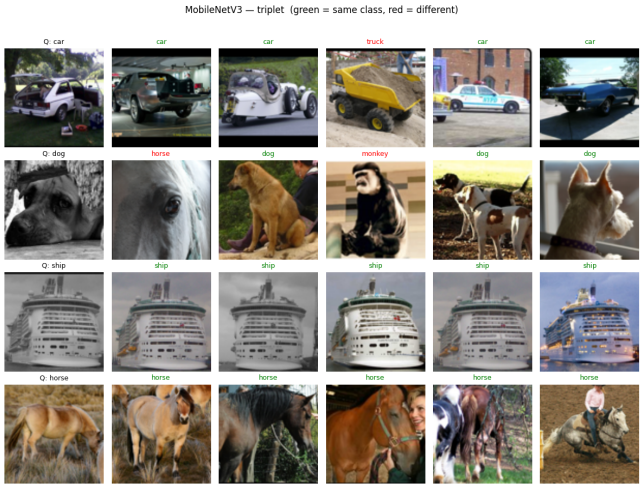
(a) ResNet-18 – classification



(b) ResNet-18 – Triplet



(c) MobileNetV3-L – classification



(d) MobileNetV3-L – Triplet

Figure 9: Qualitative retrieval results: query image (leftmost column) and Top-5 retrieved gallery images for each backbone and training objective. Green border: same class as the query. Red border: different class.

4 Conclusion

The three parts covered the full chain from a small-image classifier trained from scratch to a metric-learning retrieval system. In **Part I**, a compact VGG-style CNN reached about 90% mCA on CIFAR-10 with 2.20 M parameters (half of the baseline), +26 pp over the 64% baseline (Config 4, the selected model, scores 90.12%); the four hyperparameter configurations illustrate how learning rate, batch size and optimizer interact under the same regularisation budget. In **Part II**, scratch training on STL-10 stayed below 60% mCA while ImageNet-pretrained fine-tuning reached around 95.5%, and a middle fusion of the two pretrained backbones added only 14 730 parameters while improving the best single backbone by 0.3 pp, confirming that ResNet-18 and MobileNetV3-L make complementary errors. In **Part III**, the Part II backbones were turned into image-embedding networks; the classification embeddings already gave strong retrieval, and Triplet Loss fine-tuning ($m = 1.0$, 10 epochs) improved mAP for both backbones (+1.7 pp for ResNet-18, +3.3 pp for MobileNetV3-L) at the cost of a small drop in P@1, which shows that the best classifier is not necessarily the best image-embedding network: cross-entropy optimises the top of the softmax, whereas Triplet Loss optimises exactly the geometry evaluated at retrieval time.